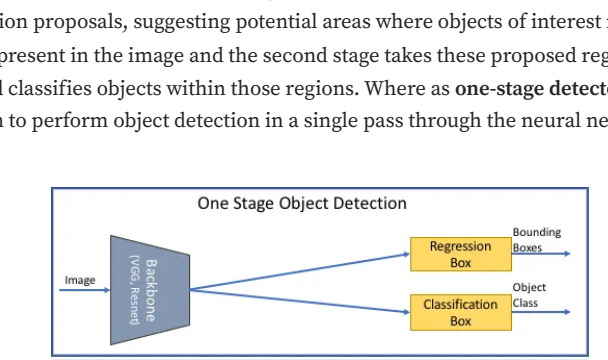
Object Detection part-2: Two Stage Detectors: R-CNN,Fast R-CNN,Faster R-CNN

Rajesh Katta · Follow · 20 min read · Mar 1, 2024

222 · 2

Object detection, the task of precisely locating and classifying objects within an image, has witnessed a paradigm shift with the introduction of **Two-Stage Detectors**. Unlike their predecessors, these detectors adopt a nuanced approach, breaking down the detection process into two distinctive stages involving region proposal and object classification. The first stage generates region proposals, suggesting potential areas where objects of interest might be present in the image and the second stage takes these proposed regions and classifies objects within those regions. Where as **one-stage detectors** aim to perform object detection in a single pass through the neural network.



In this exploration, we will unravel the intricacies of Two-Stage Detectors, delving into their architecture, key components, and the underlying mechanisms in detail.

R-CNN:

R-CNN, or Region-based Convolutional Neural Networks, introduced by Girshick et al. in 2014, represents a significant advancement in object detection within computer vision. The R-CNN pipeline can be broken down into several stages, each contributing to the overall process of detecting and classifying objects within an image.

Region Proposal Generation:

The first stage involves generating region proposals, which are bounding boxes or regions that potentially contain objects of interest within the image. Selective search is utilized in the original R-CNN paper for this purpose. Selective search employs a segmentation algorithm to create an initial set of region proposals. It iteratively merges segments of the image to form larger regions, and bounding boxes are then drawn around these merged regions.

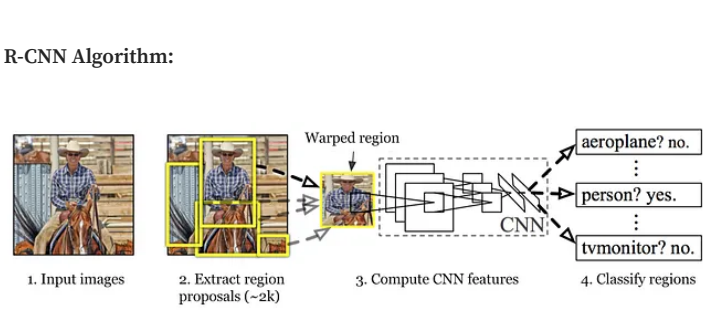
Feature Extraction from Region Proposals:

Once region proposals are generated, the second stage involves extracting features from each proposed region. A fully convolutional neural network (CNN) is applied independently to each region to compute feature representations. In the original R-CNN paper, the authors use the convolutional layers of AlexNet as the feature extractor. However, other CNN architectures can also be utilized for this purpose.

Bounding Box Refinement and Classification:

In the final stage, the extracted features from the region proposals are fed into a fully connected layer. This layer serves as a classifier, typically implemented as Support Vector Machines (SVMs) in the original R-CNN formulation. The classifier is trained to classify each region proposal into one of the predefined object categories. Additionally, bounding box regression is performed to refine the initial bounding box estimates obtained from the region proposal generation stage.

R-CNN Algorithm:



1. **Pre-train a CNN:** Start by pre-training a CNN model on an image classification task, such as VGG or ResNet, using a large dataset like ImageNet. This pre-trained model will serve as the feature extractor for region proposals.
2. **Generate Region Proposals:** Utilize some external algorithm like selective search to propose regions of interest within the image. This step generates around 2000 region proposals per image, each of varying sizes and shapes.
3. **Warp Region Proposals:** Warp each region proposal to a fixed size required by the CNN. This ensures uniformity in input size for feature extraction.
4. **Fine-tune CNN on Region Proposals:** Fine-tune the pre-trained CNN on the warped region proposals for the classification task. The CNN is trained for K + 1 classes, where K is the number of object classes, and the additional class represents the background. During fine-tuning, a smaller learning rate is used, and the mini-batch oversamples positive cases due to the predominance of background regions.
5. **Feature Extraction:** For each region proposal, perform forward propagation through the fine-tuned CNN to generate a feature vector. This vector represents the features of the proposed region.
6. **Binary SVM Classification:** Train a binary Support Vector Machine (SVM) for each class independently or one svm per class using the feature vectors generated in the previous step and train this with hing loss. Positive samples for each class are region proposals with an IoU overlap threshold greater than or equal to 0.3 with ground truth annotations, while negative samples are regions not containing any relevant object.
7. **Bounding Box Regression:** To refine localization, train a regression model using CNN features to predict bounding box correction offsets. This model corrects the predicted detection window based on the features extracted by the CNN.
8. **Detection:** During inference, for a given image, apply the trained SVMs to classify each region proposal. Overlapping region proposals are rejected using non-maximum suppression. The final step involves refining the bounding boxes using the regression model to reduce localization errors.

Bounding box regression:

Initially, R-CNN generates region proposals using selective search or another region proposal method. Each proposal is represented by its bounding box coordinates: **P=(px,py,pw,ph)**, where px and py are the center coordinates, and pw and ph are the width and height of the proposal box. For each region proposal, if there exist an object of interest, then there exists a ground truth bounding box **G=(gx,gy,gw,gh)** that tightly encloses the object of interest in the image. These ground truth boxes are provided in the training dataset. The goal of bounding box regression is to learn a transformation **d(P)** that adjusts the coordinates and dimensions of the initial region proposal **P** to better align with the ground truth bounding box **G**, based on the features of the proposal. The model learns to adjust the proposal box to better fit the ground truth box based on relative adjustments per unit dimensions of proposal box. Imagine stretching or shrinking the proposal box while its center moves, all guided by features.

It learns a function (d(P)) that maps features of a proposed box (P) to adjustments in its center coordinates and dimensions. These adjustments depend on the specific object and context captured in the features.

- `d(P)` often takes a linear form: `d(P) = w^T * Φ(P) + b`, where:
- `w` is a learned weight vector (captures adjustments pattern)
- `Φ(P)` are features extracted from the proposal box (encode object information)
- `b` is a bias term (accounts for overall shift)

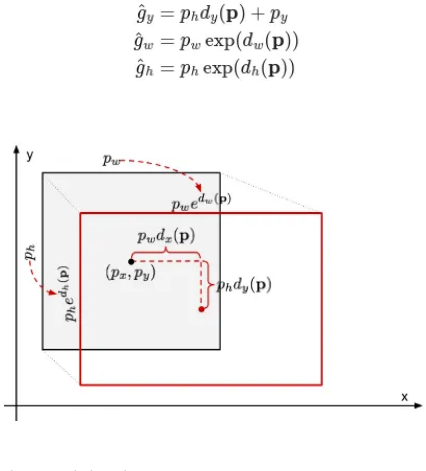
Adjustments (d(P)) include-

- `dx(P), dy(P)`: change in center coordinates relative to proposal size (per unit width/height of proposal box).
- `dw(P), dh(P)`: scaling factors for width and height adjustments based on proposal box dimensions(exponential ensures positivity).

Prediction Process:

1. Features are extracted from the proposed box.
2. `d(P)` predicts adjustments based on the learned weights and features.
3. Adjustments are added to the original proposal coordinates and dimensions.

$$\begin{aligned}\hat{g}_x &= p_w d_x(\mathbf{p}) + p_x\\ \hat{g}_y &= p_h d_y(\mathbf{p}) + p_y\\ \hat{g}_w &= p_w \exp(d_w(\mathbf{p}))\\ \hat{g}_h &= p_h \exp(d_h(\mathbf{p}))\end{aligned}$$



The targets for the model to learn are -

$$\begin{aligned}t_x &= (g_x - p_x)/p_w\\ t_y &= (g_y - p_y)/p_h\\ t_w &= \log(g_w/p_w)\\ t_h &= \log(g_h/p_h)\end{aligned}$$

The loss function for bounding box regression is the mean squared error with regularization-

$$\mathcal{L}_{\text{reg}} = \sum_{i\in\{x,y,w,h\}} (t_i - d_i(\mathbf{p}))^2 + \lambda\|\mathbf{w}\|^2$$

Training:

1. Calculate differences: We compute the changes in x, y, width, and height between proposal and ground truth boxes, normalized by the proposal's dimensions. These become the desired adjustments (targets) for the model to learn.
2. Model learns the "shift": The model takes proposal features and predicts adjustments per unit (like dx/pw) using a function (d(P)) based on learned weights and the features.

Inference:

1. Model predicts shift: Given a new proposal and its features, the model predicts dx, dy, dw, dh values.
2. Apply shift: We apply these adjustments to the proposal box to get the predicted ground truth box.

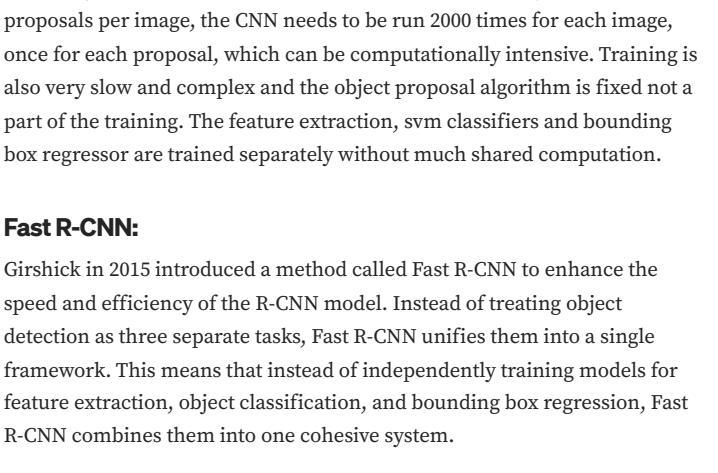
Con's of R-CNN:

In R-CNN, the Selective Search algorithm generates around 2000 region proposals, each representing a candidate area where objects of interest may be located within the image. The CNN then extracts features from each of these region proposals independently. These features are then passed through the fully connected layers to classify each proposed region independently into one of the predefined object categories plus one category for no object or background. Since there are approximately 2000 region proposals per image, the CNN needs to be run 2000 times for each image, once for each proposal, which can be computationally intensive. Training is also very slow and complex and the object proposal algorithm is fixed not a part of the training. The feature extraction, svm classifiers and bounding box regressor are trained separately without much shared computation.

Fast R-CNN:

Girshick in 2015 introduced a method called Fast R-CNN to enhance the speed and efficiency of the R-CNN model. Instead of treating object detection as three separate tasks, Fast R-CNN unifies them into a single framework. This means that instead of independently training models for feature extraction, object classification, and bounding box regression, Fast R-CNN combines them into one cohesive system.

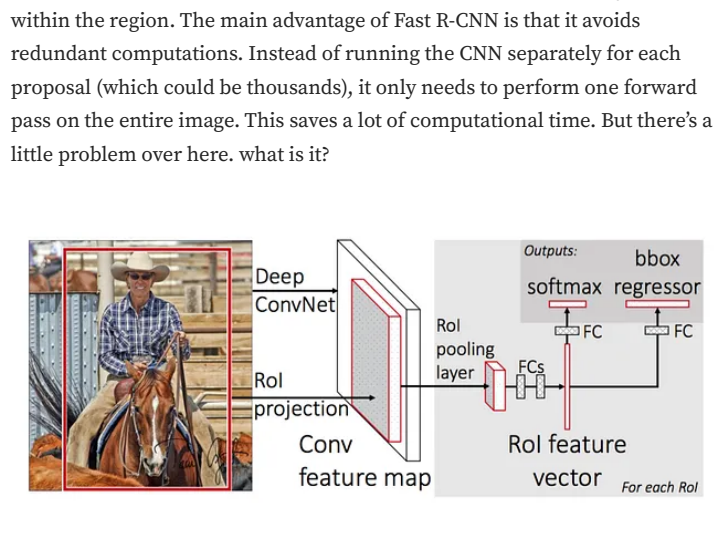
In the original R-CNN approach, CNN features were extracted separately for each region proposal, leading to redundant computations. Fast R-CNN addresses this issue by performing a single forward pass of the entire image through the convolutional neural network. Instead of running the feature extractor (CNN) separately for each region proposal, Fast R-CNN does it differently. It runs the feature extractor just once on the entire image in a single forward pass. This means it extracts all the features needed for object detection from the whole image in one go. After extracting features from the entire image, Fast R-CNN then projects the region proposals onto this feature map. Each region proposal corresponds to a specific area of interest within the image. With the region proposals mapped onto the feature map, Fast R-CNN runs both the object classifier and the bounding box regressor on these features region proposals. The classifier determines what objects are present in each region, while the regressor refines the location of these objects within the region. The main advantage of Fast R-CNN is that it avoids redundant computations. Instead of running the CNN separately for each proposal (which could be thousands), it only needs to perform one forward pass on the entire image. This saves a lot of computational time. But there's a little problem over here. what is it?



After extracting features from the entire image, we obtain a feature map and we project the region proposals on this feature map and get the portion of feature map in that region. However, the classifier and regressor accepts a fixed size input and the region proposals generated by the region proposal algorithm come in different sizes and aspect ratios. This presents a challenge because traditional neural networks require inputs of fixed sizes. To address this challenge, Fast R-CNN utilizes the concept of ROI pooling(region of interest pooling).

ROI pooling:

ROI pooling is a technique that allows us to extract fixed-size feature maps from arbitrary-sized regions of interest within the feature map. Let's see how it works-



0.1	0.2	0.3	0.4	0.5	0.6
1	0.7	0.2	0.6	0.1	0.9
0.9	0.8	0.7	0.3	0.5	0.2

ROI Pooling animation

- For each region proposal, ROI pooling divides the region into a fixed number of sections or a grid(e.g, a grid of 2x2 or 3x3).
- It then applies max pooling independently in each section to obtain fixed-size feature maps.
- These fixed-size feature maps are then passed to the classifier and regressor for further processing.
- Importantly, ROI pooling ensures that regardless of the size or aspect ratio of the region proposal, the extracted feature maps are always of the same size.
- Back propagation through ROI pooling layer is same as max pooling as it's just max pooling in different regions.

ROI pooling enables Fast R-CNN to handle region proposals of varying sizes efficiently by extracting fixed-size feature maps from arbitrary-sized regions of interest within the feature map.

Loss Function:

Beneficial intelligence
Get The Medium Newsletter for weekly wisdom.
Subscribe now

The loss function sums up the cost of classification and bounding box prediction:

$$\mathcal{L} = \mathcal{L}_{\text{cls}} + \mathcal{L}_{\text{box}}.$$

For background or no object of interest, we just ignore the loss of bounding box prediction. We can simply create an identity function to handle this-

$$\mathbb{1}[u >= 1] = \begin{cases}1 & \text{if } u \geq 1\\ 0 & \text{otherwise}\end{cases}$$

where u is the true class label. So, if there's an object u≥1, then this function returns one else returns zero. So, overall loss function becomes-

$$\mathcal{L}(p, u, t^u, v) = \mathcal{L}_{\text{cls}}(p, u) + \mathbb{1}[u \geq 1]\mathcal{L}_{\text{box}}(t^u, v)$$

where, p is the predicted class probability, u is the true class label, tu is the predicted bounding box adjustments and v is the true bounding box adjustments. The loss functions for classification and bounding box regression are cross entropy loss and mean squared error-

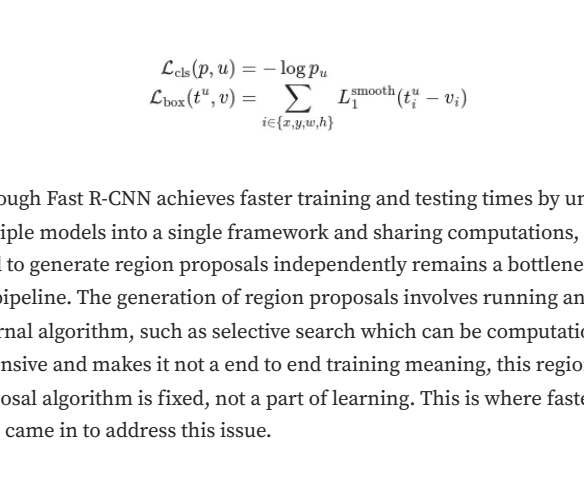
$$\begin{aligned}\mathcal{L}_{\text{cls}}(p, u) &= -\log p_u\\ \mathcal{L}_{\text{box}}(t^u, v) &= \sum_{i\in\{x,y,w,h\}} L_1^{\text{smooth}}(t_i^u - v_i)\end{aligned}$$

Although Fast R-CNN achieves faster training and testing times by unifying multiple models into a single framework and sharing computations, the need to generate region proposals independently remains a bottleneck in the pipeline. The generation of region proposals involves running an external algorithm, such as selective search which can be computationally expensive and makes it not a end to end training meaning, this region proposal algorithm is fixed, not a part of learning. This is where faster R-CNN came in to address this issue.

Faster R-CNN:

In the evolution of object detection architectures, the transition from R-CNN to Fast R-CNN marked a significant improvement in computational efficiency by incorporating the region proposal algorithm directly into the neural network. However, the region proposal stage in Fast R-CNN still relied on CPU-based algorithms like Selective Search in which no learning is involved, which incurred significant computational overhead and hindered real-time performance. The introduction of Faster R-CNN addressed this limitation by introducing the Region Proposal Network (RPN), a convolutional network that generates region proposals directly from the input image. By leveraging the RPN, Faster R-CNN drastically reduced the region proposal time from several seconds to mere milliseconds per image, leading to significant speed improvements. Moreover, by sharing layers between the RPN and subsequent detection stages, Faster R-CNN enhanced feature representation throughout the entire detection pipeline, ultimately improving the accuracy and efficiency of object detection systems.

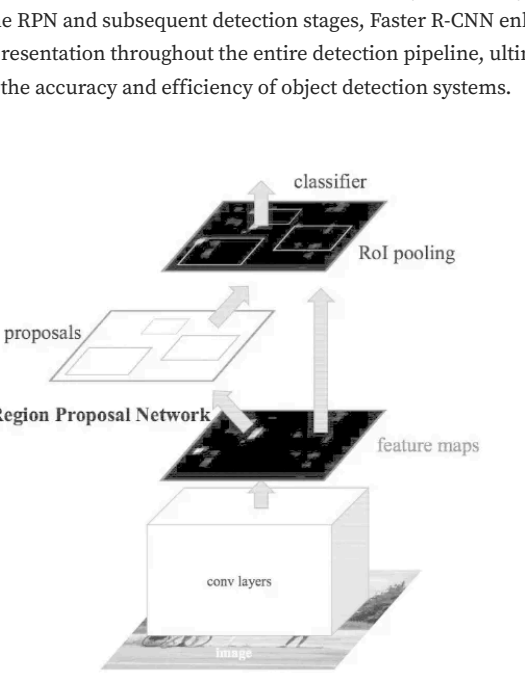
Top highlight



Faster RCNN

1. Backbone:

The first step in the Faster R-CNN architecture is same as the previous versions utilizing a pretrained Convolutional Neural Network (CNN), such as VGG or ResNet, to extract informative features from input images, utilizing an intermediate convolutional layer's output for subsequent object detection tasks. Each convolutional layer in the CNN learns hierarchical features from the input image. Lower layers detect simple features like edges, while deeper layers capture complex patterns and object representations. Through convolution and pooling operations, the input image is transformed into a convolutional feature map with reduced spatial dimensions but increased depth. The convolutional feature map retains important spatial relationships between objects and features. Each location in the map corresponds to a receptive field in the original image, capturing localized information. For example, if the feature map of an image of size 224x224 is 7x7, then each 1x1 location on the feature map is corresponds to a 32x32 sized receptive field at the corresponding location on the image. Despite spatial dimension reduction, the feature map preserves the relative positions of objects and features encoded by the network. While VGG was widely used in earlier implementations of Faster R-CNN, architectures like ResNet have become more common due to their superior performance and efficiency. Despite differences in architecture, both VGG and ResNet serve the same purpose in Faster R-CNN: to extract informative features from input images, which are then used for region proposal and object detection tasks.



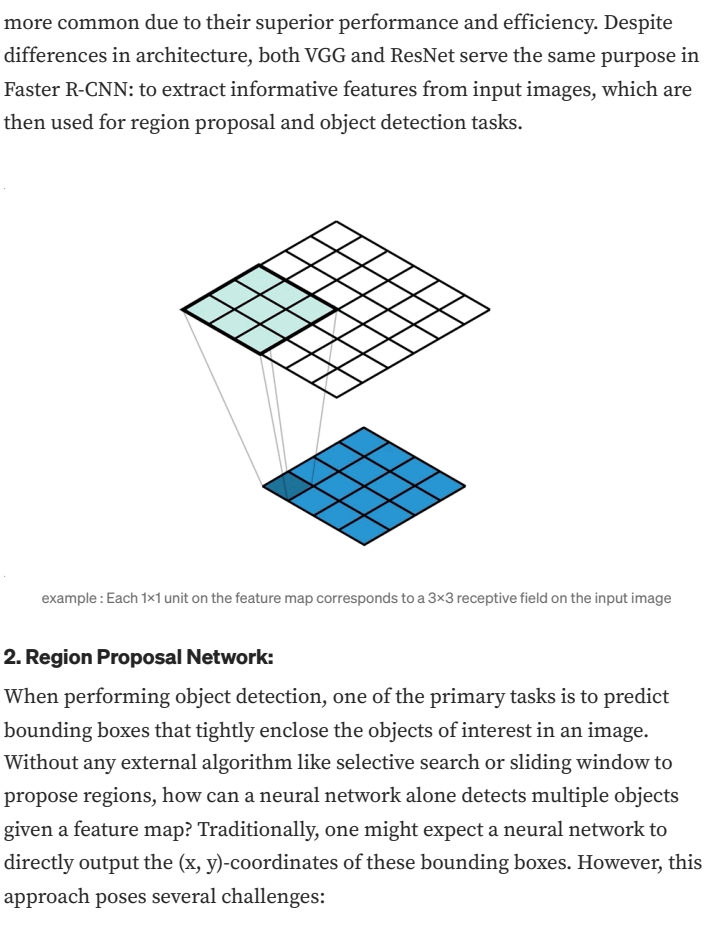
example : Each 1×1 unit on the feature map corresponds to a 3×3 receptive field on the input image

2. Region Proposal Network:

When performing object detection, one of the primary tasks is to predict bounding boxes that tightly enclose the objects of interest in an image. Without any external algorithm like selective search or sliding window to propose regions, how can a neural network alone detects multiple objects given a feature map? Traditionally, one might expect a neural network to directly output the (x, y)-coordinates of these bounding boxes. However, this approach poses several challenges:

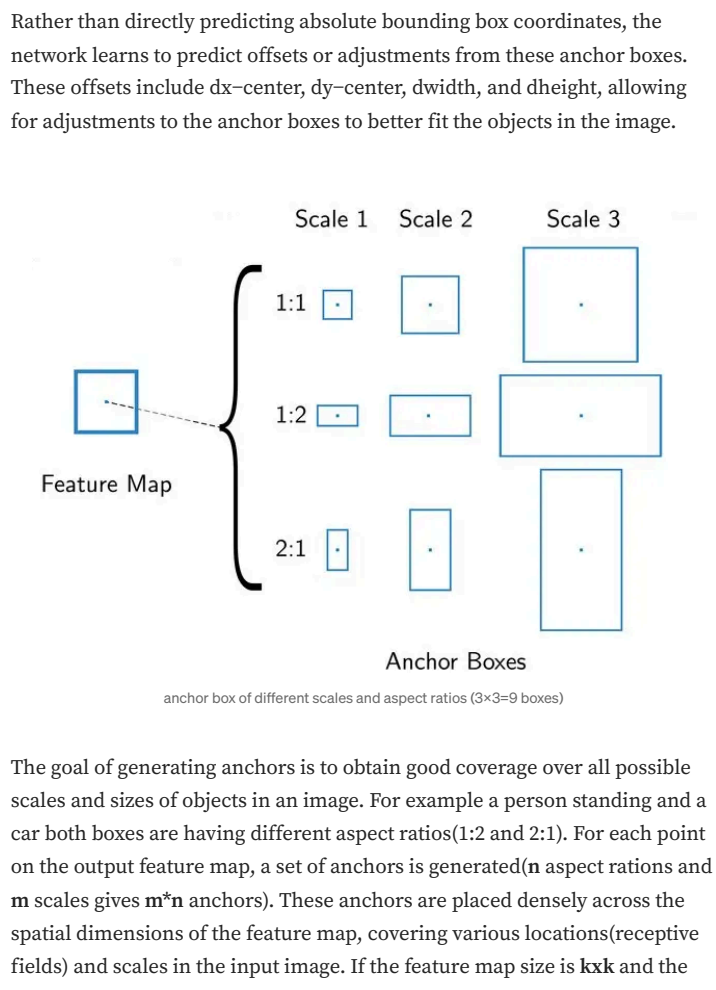
1. Directly predicting bounding box coordinates might lead to values that fall outside the boundaries of the image. This creates difficulties in interpretation and application since bounding boxes must remain within the image limits.
2. Ensuring that predicted bounding boxes satisfy constraints like **xmin < xmax and ymin < ymax** is essential for valid bounding box predictions. However, directly predicting coordinates doesn't inherently enforce these constraints, potentially leading to invalid box predictions.

To address these challenges, Girshick et al. proposed the concept of **anchors** in the Faster R-CNN framework. Anchors are predefined bounding boxes of various sizes and aspect ratios that act as reference boxes across the image. Rather than directly predicting absolute bounding box coordinates, the network learns to predict offsets or adjustments from these anchor boxes. These offsets include dx-center, dy-center, dwidth, and dheight, allowing for adjustments to the anchor boxes to better fit the objects in the image.



anchor box of different scales and aspect ratios (3×3=9 boxes)

The goal of generating anchors is to obtain good coverage over all possible scales and sizes of objects in an image. For example a person standing and a car both boxes are having different aspect ratios(1:2 and 2:1). For each point on the output feature map, a set of anchors is generated(**n** aspect rations and **m** scales gives **m*n** anchors). These anchors are placed densely across the spatial dimensions of the feature map, covering various locations(receptive fields) and scales in the input image. If the feature map size is **kxk** and the number of anchor boxes are lets say **p** boxes, then it generates these **p** boxes at each location on the feature map and that gives **k*k*p** anchor boxes per image. The purpose of using anchors is to provide a systematic way of proposing potential object regions without considering all possible bounding box locations. By placing anchors of different sizes and aspect ratios at each location on the feaure map, the network can learn to predict whether an object is present at that receptive field on the image and estimate its size.



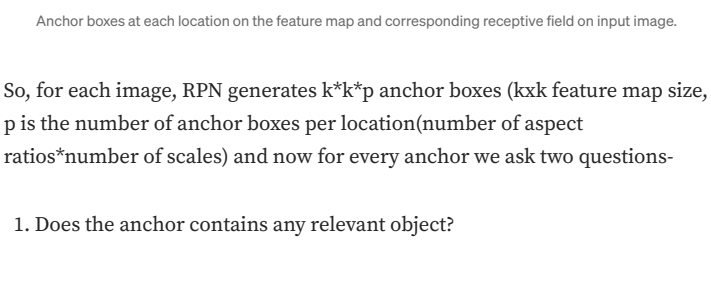
Anchor boxes at each location on the feature map and corresponding receptive field on input image.

So, for each image, RPN generates k*k*p anchor boxes (kxk feature map size, p is the number of anchor boxes per location(number of aspect ratios*number of scales) and now for every anchor we ask two questions-

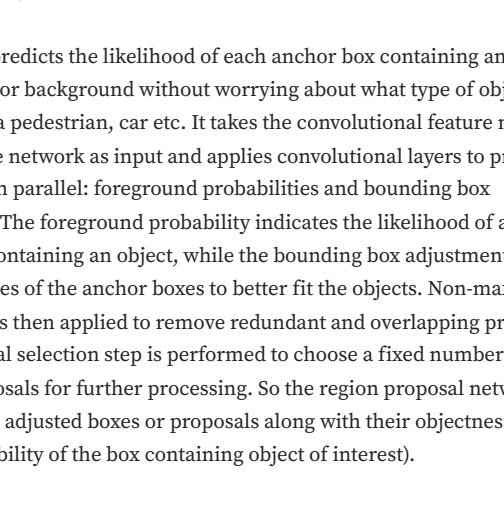
1. Does the anchor contains any relevant object?

2. If object is present, what are the adjustments to adjust that anchor box to better fit the object?

So, the RPN predicts the likelihood of each anchor box containing an object (foreground) or background without worrying about what type of object in that box like a pedestrian, car etc. It takes the convolutional feature map from the base network as input and applies convolutional layers to predict two outputs in parallel: foreground probabilities and bounding box adjustments. The foreground probability indicates the likelihood of an anchor box containing an object, while the bounding box adjustments refine the coordinates of the anchor boxes to better fit the objects. Non-maximum suppression is then applied to remove redundant and overlapping proposals, and a proposal selection step is performed to choose a fixed number of top-scoring proposals for further processing. So the region proposal network outputs these adjusted boxes or proposals along with their objectness scores(probability of the box containing object of interest).



RPN Network Architecture and training:



Region proposal network

1. The RPN takes the feature map obtained from the backbone network as input, applies a 3x3 convolutional layer with 512 units. This convolutional layer operates independently at each spatial location in the feature map, producing a 512-dimensional feature vector or descriptor for every location. This step helps to enhance the representational capacity of the features and extract more discriminative information relevant for object detection.
2. After the convolutional layer, the feature map is fed into two sibling layers. The first sibling is a 1x1 convolutional layer with 18 units, which is responsible for object classification. The output of this layer has a size of (H, W, 18), where H and W represent the dimensions of the feature map and each of the 18 units in this output corresponds to one of the 9 anchors (each anchor has 2 units for objectness score, foreground or background). This output provides probabilities indicating whether each anchor at every spatial location contains an object or background.
3. The second sibling layer is another 1x1 convolutional layer, but with 36 units. This layer is responsible for bounding box regression, which aims to refine the coordinates of the anchors to better fit the objects present in the image. The output of this layer has a size of (H, W, 36), where the 36 units corresponds 9 anchors with 4 units for each (9*4=36) . The output provides regression adjustments that adjust the coordinates (x, y, width, height) of the anchors.

Training: During training, the RPN aims to effectively classify anchors as foreground (containing objects) or background (not containing objects), and to regress the bounding box coordinates of the anchors to better fit the objects.

1. All anchors are categorized into two groups based on their overlap with ground-truth objects. Anchors with an highest Intersection over Union (IoU) with the ground truth box or greater than 0.7 with any ground-truth object box are labeled as "foreground" (positive anchors), indicating they contain objects. Anchors with an IoU less than 0.3 with all ground-truth objects are labeled as "background" (negative anchors), indicating they do not contain objects. Same ground truth box can cause multiple positive anchors.
2. From the pool of labeled anchors of an image, a mini-batch of size 256 (128 foreground,128 background) is randomly sampled for training. The goal is to maintain a balanced ratio between foreground and background anchors in the mini-batch other wise it causes bais in the training towards negatives. This balanced sampling ensures that the model is exposed to sufficient examples of both classes during training, preventing class imbalance issues. If there are insufficient number of positives, then it pads the remaining in the mini batch with negatives samples.
3. Once the mini-batch is formed, the RPN training loss is calculated two types of losses:

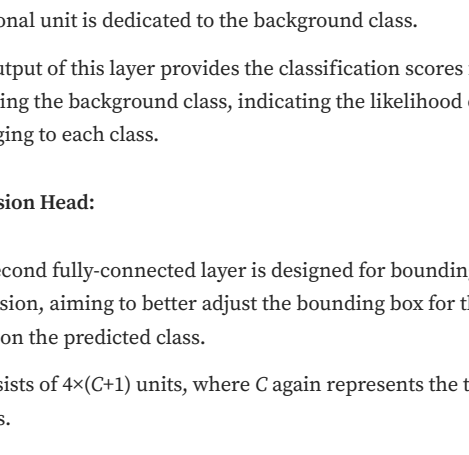
$$L(\{p_i\}, \{t_i\}) = \frac{1}{N_{cls}}\sum_i L_{cls}(p_i, p_i^*) + \lambda\frac{1}{N_{reg}}\sum_i p_i^* L_{reg}(t_i, t_i^*).$$

- Binary Classification Loss($L_{cls}(p_i, p_i^*)$): This loss is calculated using binary cross-entropy, which measures the discrepancy between predicted probabilities and ground-truth labels for foreground vs. background classification. All anchors in the mini-batch contribute to this loss calculation.
- Bounding Box Regression Loss($L_{reg}(t_i, t_i^*)$): Only foreground anchors contribute to this loss calculation. For each foreground anchor, the RPN computes the regression target by comparing its coordinates to the closest ground-truth object's coordinates. The regression loss is then calculated using Smooth L1 loss, which is a modified version of L1 loss that handles outliers better and ensures faster convergence when the errors are small.

During the testing phase of the object detection pipeline the Region Proposal Network (RPN) utilizes a series of post-processing steps and are applied to the 20,000 anchors generated for each image to obtain object proposal bounding boxes. First, the regression coefficients obtained from the RPN are applied to the anchors to refine their positions, resulting in more precise bounding boxes. Next, the bounding boxes are sorted based on their classification scores, and a non-maximum suppression (NMS) algorithm is employed with a threshold of 0.7. This process involves discarding overlapping bounding boxes by retaining only the highest-scoring box within a group of overlapping ones. As a result, approximately 2,000 proposals per image are retained after NMS. Furthermore, any bounding boxes extending beyond the image boundaries are kept intact but clipped to fit within the image boundaries.

During the training of subsequent stages, such as the Fast R-CNN detection pipeline, all 2,000 proposals generated by the RPN are utilized. However, at test time, only the top N proposals with the highest scores are selected for further processing by the Fast R-CNN detector. This selection mechanism helps streamline the detection process by focusing on the most promising object proposals while discarding less relevant ones, ultimately leading to more efficient and accurate object detection.

3. Region of Interest (RoI) Pooling:



Once the RPN generates region proposals, just like fast rcnn, they are passed through RoI pooling. RoI pooling extracts fixed-length feature vectors from each region proposal, regardless of their size or aspect ratio. This step ensures that each proposed region is represented by a consistent feature representation, facilitating subsequent processing.

4. RCNN Module:

In the final stage of Faster R-CNN, the goal is to classify object proposals into different classes and refine their bounding boxes based on the predicted class. To achieve this, R-CNN module utilizes two different fully-connected layers for each object proposal:

1. **Classification Head:**

- The first fully-connected layer is responsible for classifying the proposals into one of the possible classes, along with a background class for rejecting undesirable proposals.
- It consists of C+1 units, where C is the total number of classes, and an additional unit is dedicated to the background class.
- The output of this layer provides the classification scores for each class, including the background class, indicating the likelihood of the proposal belonging to each class.

2. **Regression Head:**

- The second fully-connected layer is designed for bounding box regression, aiming to better adjust the bounding box for the proposal based on the predicted class.
- It consists of 4×(C+1) units, where C again represents the total number of classes.
- The output of this layer comprises 4×(C+1) values, where each group of four values corresponds to the regression adjustments for one class. These adjustments are applied to refine the bounding box coordinates for each class prediction.

By using these two separate fully-connected layers, R-CNN can simultaneously perform classification and regression tasks for each object proposal, leading to more accurate and robust object detection results. The targets for this stage are calculated similarly to the Region Proposal Network (RPN), but with consideration for multiple classes. Proposals with IoU greater than 0.7 with any ground truth box are assigned to that ground truth class, while those with IoU less than 0.3 are labeled as background. Proposals without any intersection are ignored, assuming the focus is on solving harder cases. Bounding box regression targets are computed as the offset between the proposal and its corresponding ground-truth box for assigned class proposals.

For training, a balanced mini batch of size 64 is sampled, with up to 25% foreground proposals (with class) and 75% background. Classification loss is calculated using multiclass cross-entropy for all selected proposals, while Smooth L1 loss is applied to the 25% proposals matched to a ground truth box. It's crucial to handle the bounding box regression loss carefully, as the R-CNN's fully connected network outputs predictions for each class. Hence, when computing the loss, only the prediction for the correct class should be considered. This approach ensures effective training of the R-CNN model for accurate object detection and bounding box refinement.

Four Step alternte training:

In the four-step training method used by the authors to ensure weight sharing between the Region Proposal Network (RPN) and the detector network in Faster R-CNN, the process unfolds as follows:

a) Independent Training of RPN: The RPN is trained separately, following the standard procedure described earlier. The backbone CNN for this task is initialized with weights pre-trained on an ImageNet classification task, and then fine-tuned specifically for the region proposal task. This step allows the RPN to learn to propose regions of interest effectively.

b) Independent Training of Fast R-CNN Detector: Similarly, the Fast R-CNN detector network is trained independently. The backbone CNN is initialized with weights from a network pre-trained on ImageNet for classification and then fine-tuned for the object detection task. During this stage, the weights of the RPN are kept fixed, and the proposals generated by the RPN are used to train the Fast R-CNN. This phase enables the detector network to learn object classification and bounding box regression tasks.

c) Fine-tuning RPN with Fast R-CNN Weights: In this step, the RPN is re-initialized with weights from the trained Fast R-CNN network. The RPN is then fine-tuned for the region proposal task. However, only the layers unique to the RPN are fine-tuned, while the weights of the common layers shared with the detector network are kept fixed. This ensures that the RPN learns to generate region proposals using the features learned by the detector network.

d) Fine-tuning Fast R-CNN Detector with Updated RPN: Finally, using the newly fine-tuned RPN, the Fast R-CNN detector network is fine-tuned again. Similar to the previous step, only the layers unique to the detector network are fine-tuned, while the common layer weights shared with the RPN remain fixed. This step allows the detector network to adapt to the updated region proposals generated by the refined RPN while preserving the shared feature representation learned during previous training stages.

References:

[1]. https://arxiv.org/pdf/1506.01497.pdf

[2]. https://tryolabs.com/blog/2018/01/18/faster-r-cnn-down-the-rabbit-hole-of-modern-object-detection

[3]. https://towardsdatascience.com/faster-r-cnn-for-object-detection-a-technical-summary-474c5b857b46

Checkout my implementation of Faster-RCNN from scracth:

GitHub - RajeshKatta277/Faster-RCNN_From_Scratch: A simplified implementation of Faster-RCNN from...
A simplified implementation of Faster-RCNN from scratch and training it for brain tumor detection ...
github.com

Deep Learning · Object Detection · Cnn · Artificial Intelligence · Computer Vision

222 · 2

Written by Rajesh Katta

121 followers · 4 following

Enthusiastic about mathematics and AI, I delve into the depths of machine learning's mathematical core.

Follow

Responses (2)

Gabrielsferrante

What are your thoughts?

Rajesh Katta · Author
Dec 6, 2024

Great question! Two-stage detectors like Faster R-CNN have set benchmarks in accuracy, especially for tasks requiring high precision. But with the rise of one-stage detectors like YOLO and SSD, which excel in speed and simplicity, the landscape is... more

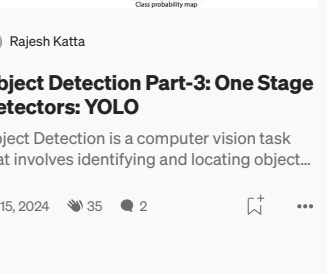
Reply

Masood Asif
Nov 18, 2024

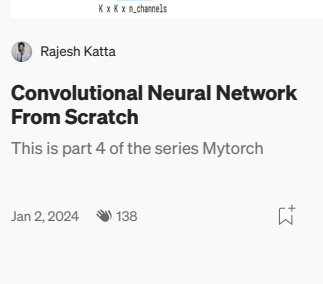
Engaging and Inquisitive: "This is a great overview of two-stage detectors. How do you see these models evolving in the future, especially considering the rise of one-stage detectors like YOLO and SSD?

Reply

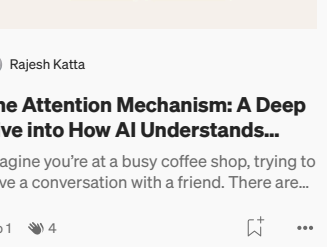
More from Rajesh Katta



Rajesh Katta
Object Detection Part-3: One Stage Detectors: YOLO
Object Detection is a computer vision task that involves identifying and locating object...
Jul 15, 2024 · 35 · 2



Rajesh Katta
Convolutional Neural Network From Scratch
This is part 4 of the series Mytorch
Jan 2, 2024 · 138



Rajesh Katta
The Attention Mechanism: A Deep Dive into How AI Understands...
Imagine you're at a busy coffee shop, trying to have a conversation with a friend. There are...
Feb 1 · 4

Rajesh Katta
Activation Functions In Depth
Part 5 of the series Mytorch.
Jan 3, 2024 · 83 · 1

See all from Rajesh Katta

Recommended from Medium

In Generative AI by Adham Khaled
Stanford Just Killed Prompt Engineering With 8 Words (And I...
ChatGPT keeps giving you the same boring response? This new technique unlocks 2×...
Oct 19, 2025 · 23K · 616

In Object Detection tutorials by Eran Feit
How to Use EigenCAM for YOLOv5 Object Detection
Understanding EigenCAM in Deep Learning
Nov 20, 2025

Michael Orozco-Fletcher
OCR Scenario 3: How Well Can Tesseract Read Documents?
Sep 10, 2025 · 55 · 1

KararHaiderDeepVision
Meet YOLOv8: The Modern Brain of Real-Time Object Detection
What is YOLOv8?
Nov 5, 2025

Mohit Gaikwad
Cracking YOLOv8: Part 5—The Conclusion: Data Flow, Inputs &...
In the final part of our series on YOLOv8, we'll walk through how data flows through the...
Sep 15, 2025 · 4

Yasserelhaddar
Faster R-CNN: Region-Based Object Detection Explained
Object detection, the task of identifying and localizing objects within an image, is a...
Nov 29, 2025 · 1

See more recommendations